

2024年度
機械学習
第2週

担当教員 小林 邦和

- 第1回 ガイダンス:機械学習の概要
- **第2回 線形モデルによるクラス分類(線形サポートベクトルマシン)**
- 第3回 非線形モデルによるクラス分類1(非線形サポートベクトルマシン)
- 第4回 非線形モデルによるクラス分類2(ニューラルネットワーク)
- 第5回 線形モデルによる回帰(線形サポートベクトル回帰)
- 第6回 非線形モデルによる回帰(非線形サポートベクトル回帰)
- 第7回 課題1(クラス分類と回帰)
- 第8回 線形モデルによる次元削減(主成分分析)
- 第9回 非線形モデルによる次元削減(オートエンコーダ)
- 第10回 非階層的クラスタリング(k平均法)
- 第11回 階層的クラスタリング(凝集クラスタリング)
- 第12回 課題2(次元削減とクラスタリング)
- 第13回 強化学習(Q学習法)
- 第14回 課題3(強化学習)
- 第15回 期末試験

- 教師あり学習(復習)

- 線形モデル

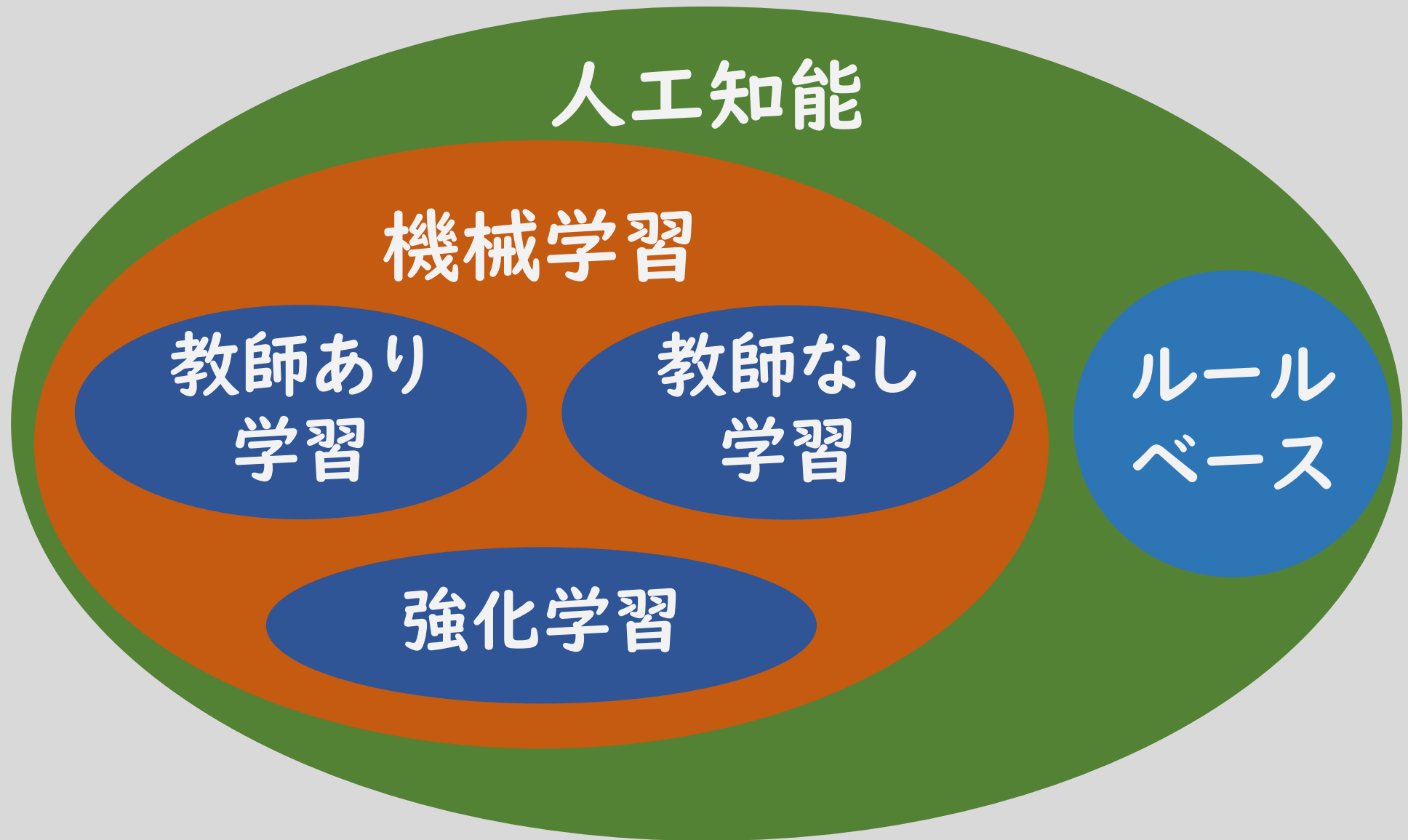
Support Vector Machine
サポートベクトルマシン

- 線形SVM

- ハードマージン線形SVM

- ソフトマージン線形SVM

- 演習(線形SVMによるクラス分類)

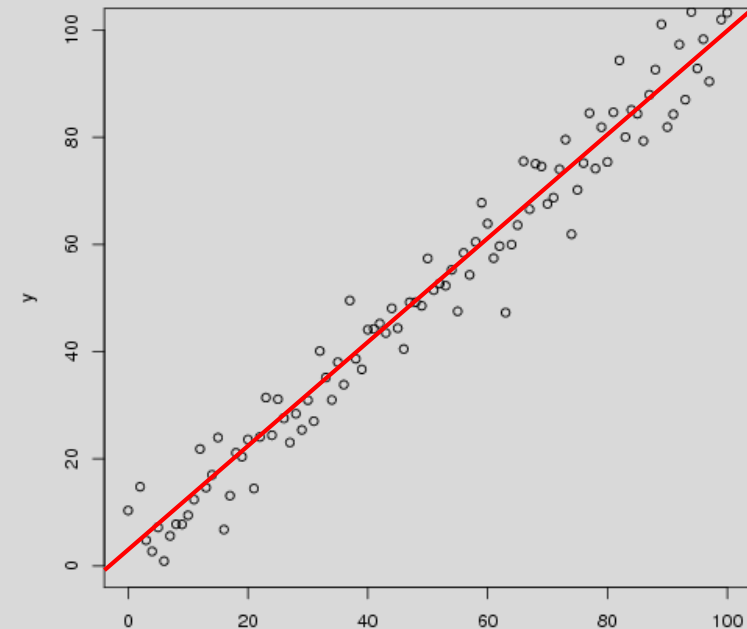


- 目的

訓練集合が、入力値とそれに対応する出力値(目標値)の事例で構成される問題を解く

- 適用例

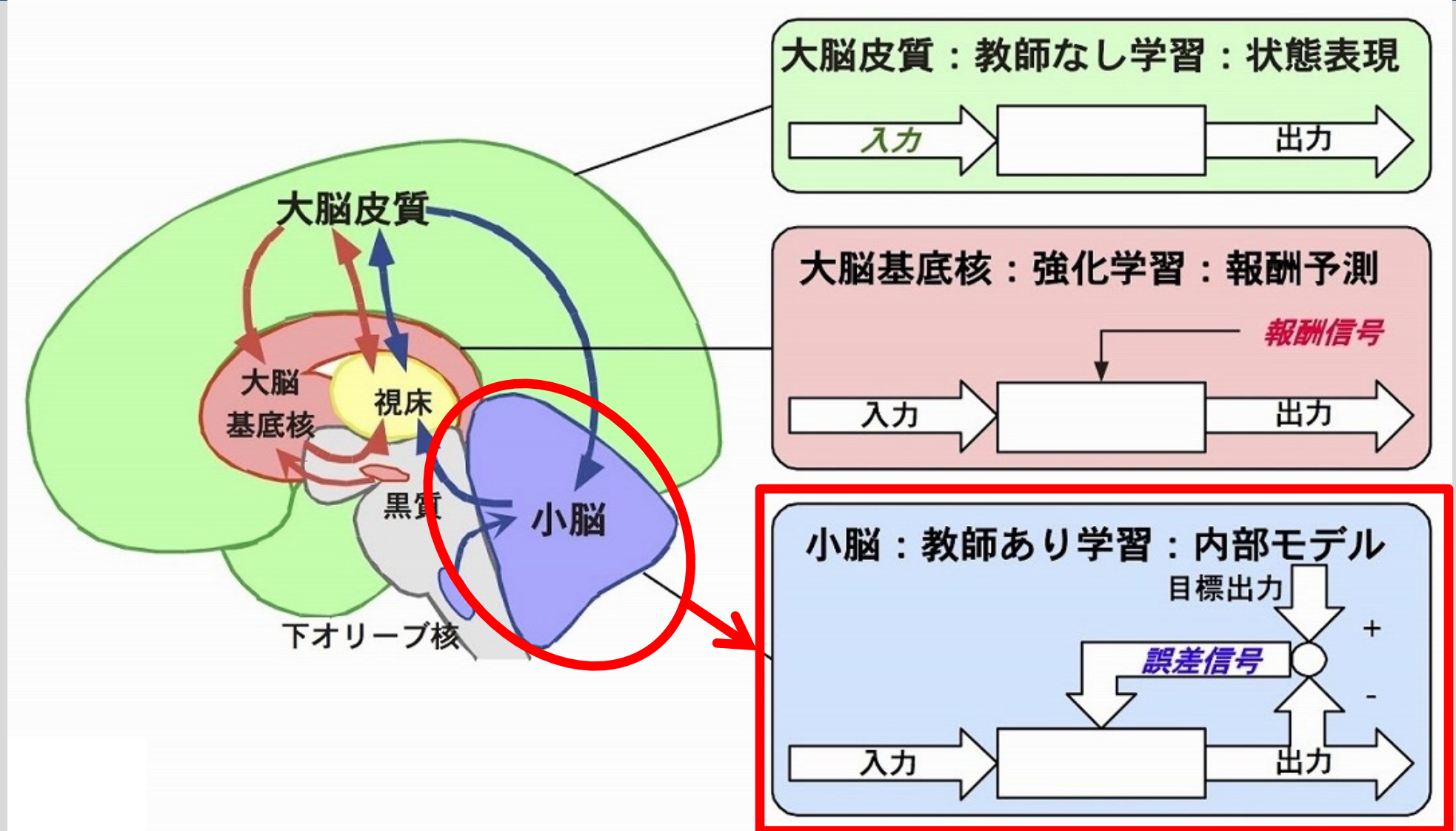
- 回帰
- クラス分類



与えられたデータ群をよく表す線を引く問題

人間は小脳で教師あり学習している！

92



出典：「脳の各器官ごとの学習アルゴリズムによる機能分化」(銅谷, 1999)

線形モデル

予測式: $\hat{y} = h(x; w, b)$

$$= w^T x + b$$

$$= w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

決定関数: $h(x; w, b)$

パラメータベクトル: $w = (w_1, w_2, \cdots, w_n)^T$

特徴量ベクトル: $x = (x_1, x_2, \cdots, x_n)^T$

バイアス成分(スカラー): b

$$\begin{aligned}\text{予測式: } \hat{y} &= h(\mathbf{x}; \mathbf{w}, b) \\ &= \mathbf{w}^T \mathbf{x} + b \\ &= w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b\end{aligned}$$

予測 $\hat{y}$ は連続値なので、2クラス分類へ適用するために
離散化が必要

2クラス分類:

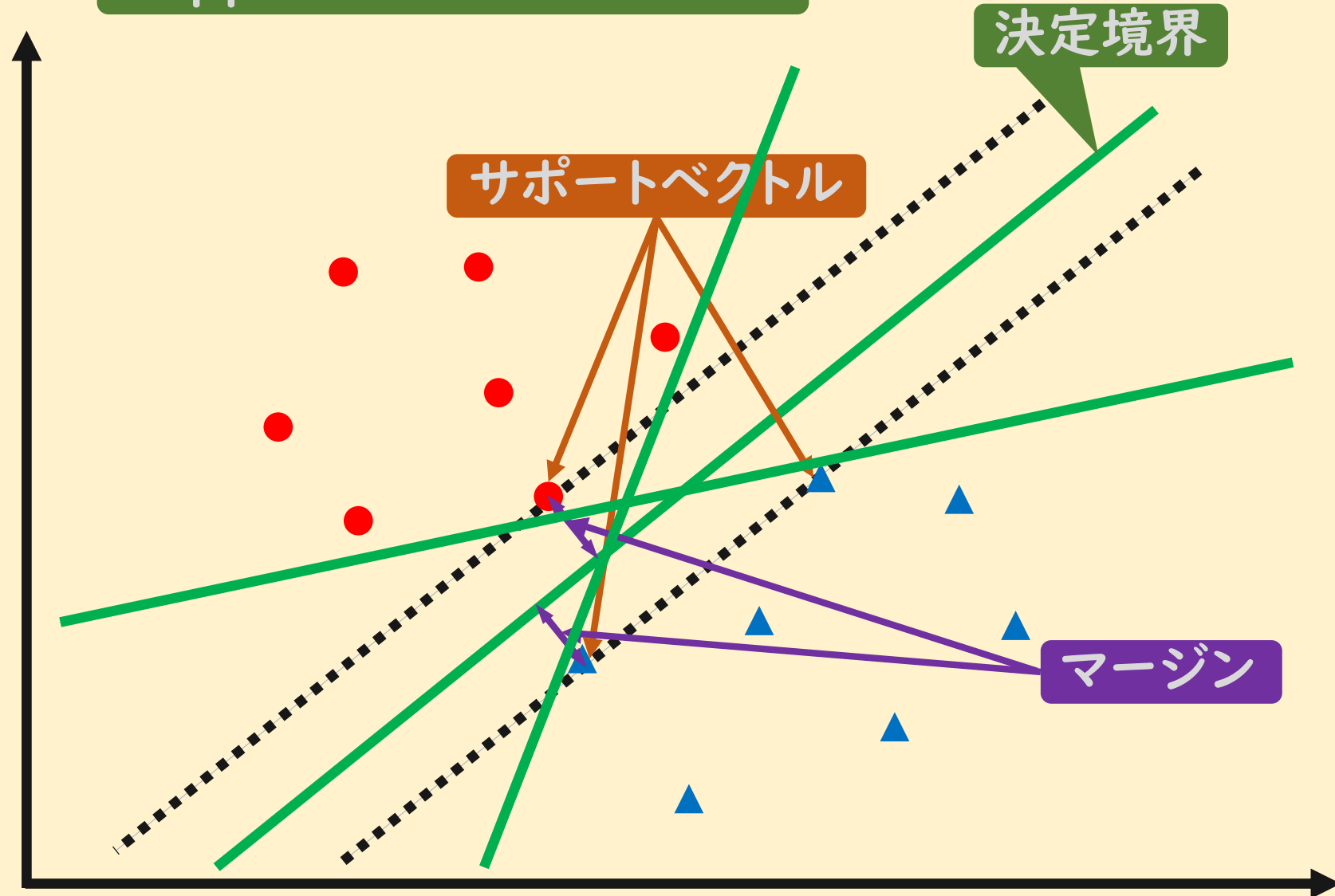
$$\hat{y} = \begin{cases} +1 & \text{if } h(\mathbf{x}; \mathbf{w}, b) \geq 0 \\ -1 & \text{if } h(\mathbf{x}; \mathbf{w}, b) < 0 \end{cases}$$

線形SVM

線形SVMの概念

97

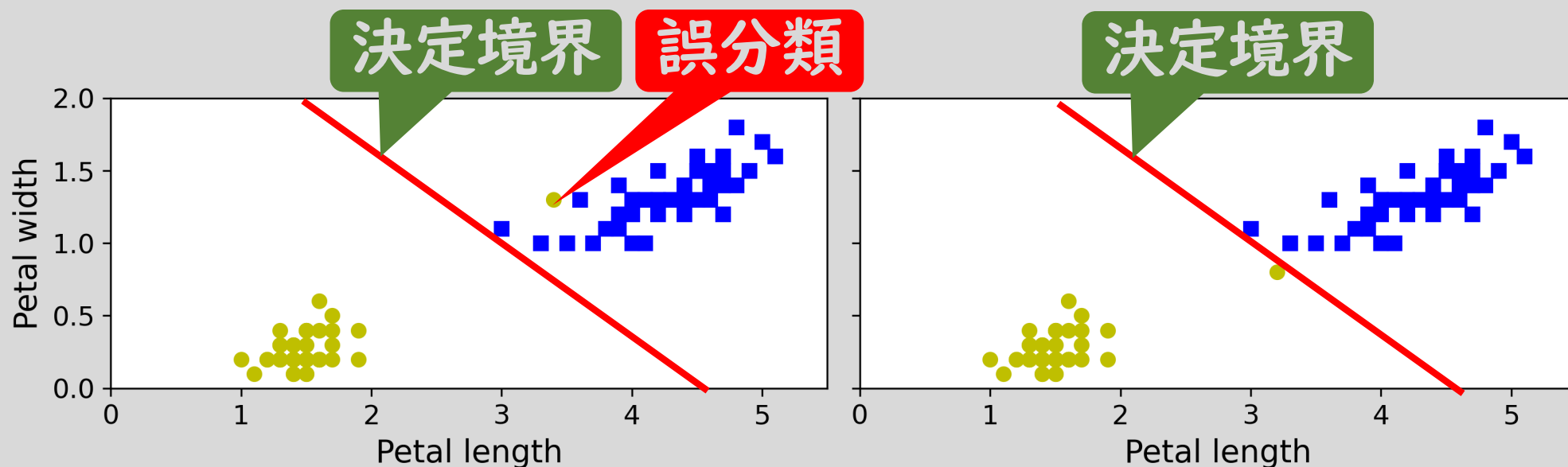
Support Vector Machine



線形分離可能性

98

- 2次元特徴量は**直線**, 3次元特徴量は**平面**, 4次元以上の特徴量は**超平面**で2つのクラスを分離できる



誤分類あり
→ **線形分離不可能**

誤分類なし
→ **線形分離可能**

- **ハードマージン型**

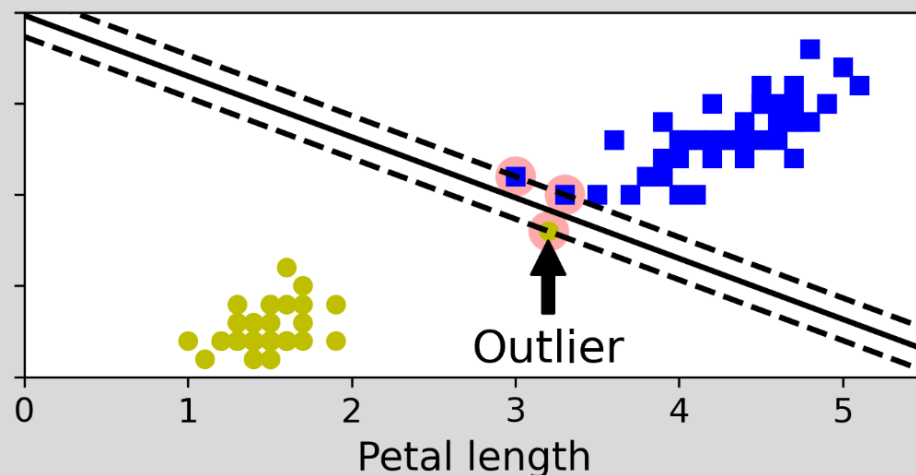
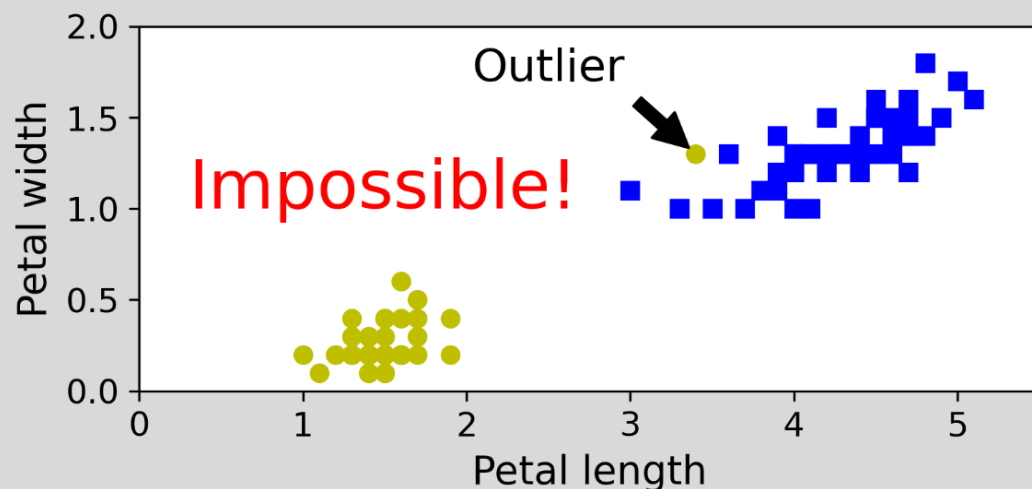
線形分離可能なデータ集合に対してのみ適用可能

→ 外れ値がある場合, 適用不可(左下図)

- **ソフトマージン型**

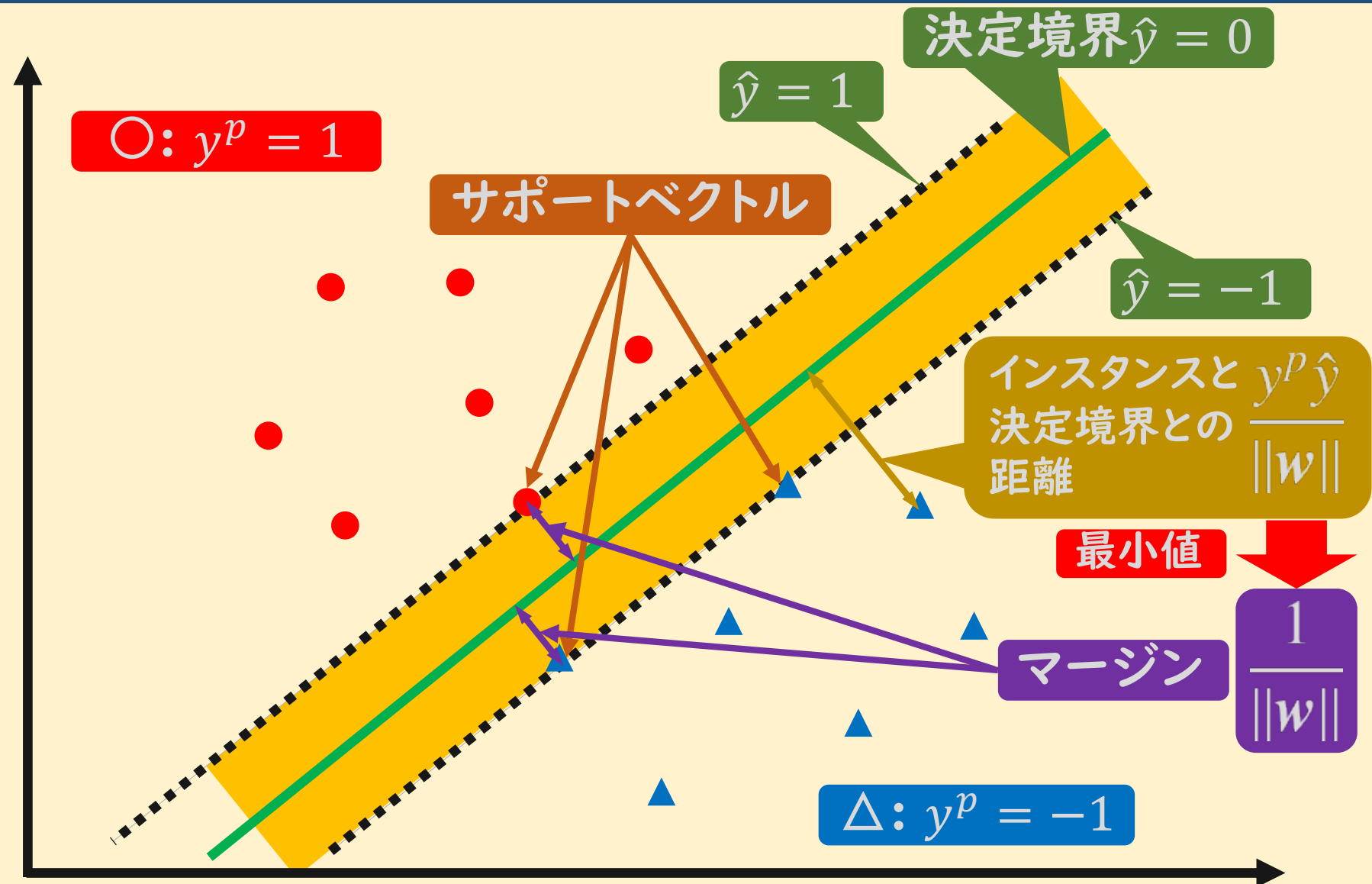
線形分離不可能なデータ集合に対しても適用可能

→ 外れ値がある場合でも, 適用可能



SVMの訓練目的(マージンの最大化)

100



インスタンスが一つも含まれない最も幅広の道を境界に通す

SVMのマージン最大化

101

マージンの定義: $\min_p \frac{y^p \hat{y}}{\|w\|}$

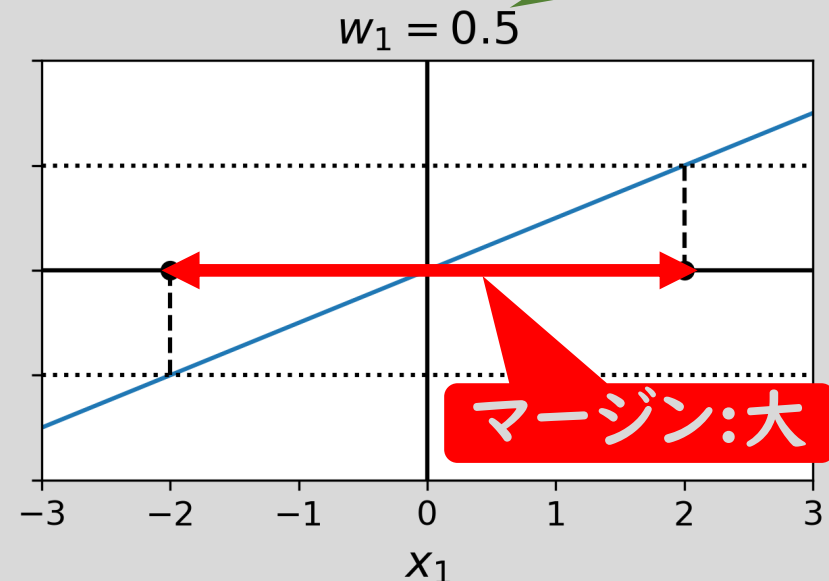
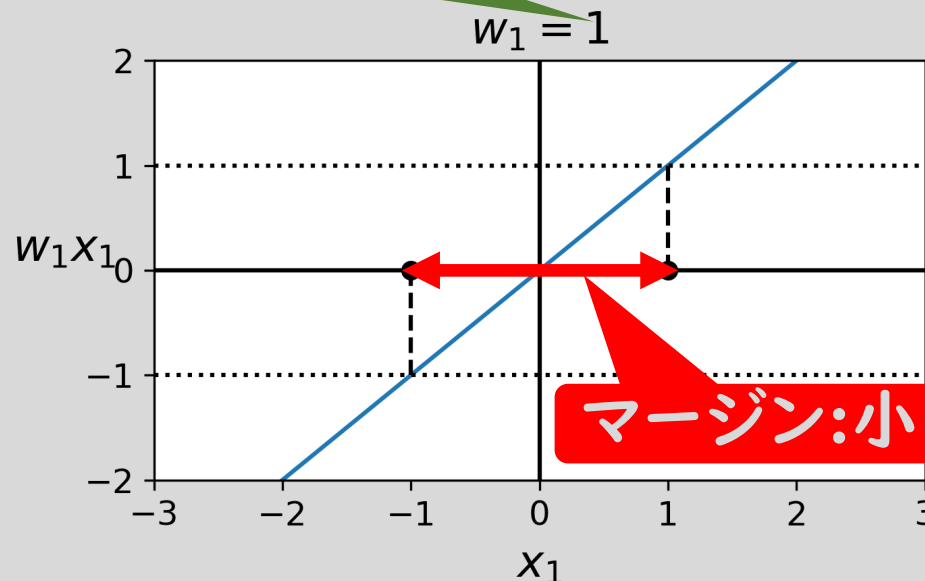
x^p と決定境界との距離

マージン最大化

$$\max_{w,b} \min_p \frac{y^p \hat{y}}{\|w\|} = \max_{w,b} \frac{1}{\|w\|} \min_p y^p \hat{y} = \max_{w,b} \frac{1}{\|w\|}$$

パラメータ(傾き):大

パラメータ:小



パラメータ w_1 が小さいほど, マージンが大きくなる

ハードマージン線形SVM

マージン最大化(ハードマージン線形SVM)

103

マージン:

$$\frac{1}{\|\mathbf{w}\|}$$

最適化問題:

$$\max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|} \Leftrightarrow \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

マージン最大化 $\rightarrow$ ノルムの最小化

制約条件付き最適化:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

制約条件: マージン内に
インスタンスが存在しない

$$\text{subject to } y^p (\mathbf{w}^T \mathbf{x}^p + b) \geq 1 \quad (p = 1 \sim N)$$

制約条件付き最適化問題

→ ラグランジュ関数 L を導入して**双対問題**へ

関数 L の最小化問題

$$L(\mathbf{w}, b, \mathbf{a}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{p=1}^N a_p (y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1)$$

ここで, ラグランジュ乗数ベクトル

$$\mathbf{a} = (a_1, a_2, \dots, a_N)^T$$

$$a_p \geq 0 \quad (p = 1 \sim N)$$

KKT条件(ハードマージン線形SVM)

105

Karush-Kuhn-Tucker

ラグランジュ未定乗数法の解が存在するための条件($p = 1 \sim N$)

$$\begin{cases} a_p \geq 0 \\ y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1 \geq 0 \\ a_p \{y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1\} = 0 \end{cases}$$

サポートベクトル $\mathbf{x}^p$ では1番目と2番目の式, それ以外の $\mathbf{x}^p$ では $a_p = 0$ となり3番目の式が成立

問題解法(ハードマージン線形SVM)[1/3]

106

ラグランジュ関数 L を w, b で各々偏微分し, ゼロとおくと,

$$\frac{\partial L}{\partial w} = w - \sum_{p=1}^N a_p y^p x^p = \mathbf{0}$$

$$\frac{\partial L}{\partial b} = - \sum_{p=1}^N a_p y^p = 0$$

従って,

$$\mathbf{w} = \sum_{p=1}^N a_p y^p \mathbf{x}^p$$

$$\sum_{p=1}^N a_p y^p = 0$$

これら関数 L へ代入すると,

$$\begin{aligned} L(\mathbf{w}, b, \mathbf{a}) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{p=1}^N a_p (y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1) \\ &= \frac{1}{2} \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q \\ &\quad - \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q - \sum_{p=1}^N a_p y^p b + \sum_{p=1}^N a_p \\ &= -\frac{1}{2} \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q + \sum_{p=1}^N a_p \end{aligned}$$

従って, 関数 L の最小化問題(双対問題)は, 次式となる

$$\min_a \left(-\frac{1}{2} \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q + \sum_{p=1}^N a_p \right)$$

subject to

$$\sum_{p=1}^N a_p y^p = 0$$

$$a_p \geq 0$$

$$a_p \left\{ y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1 \right\} = 0 \quad (p = 1 \sim N)$$

最適解(ハードマージン線形SVM)

110

最適解:

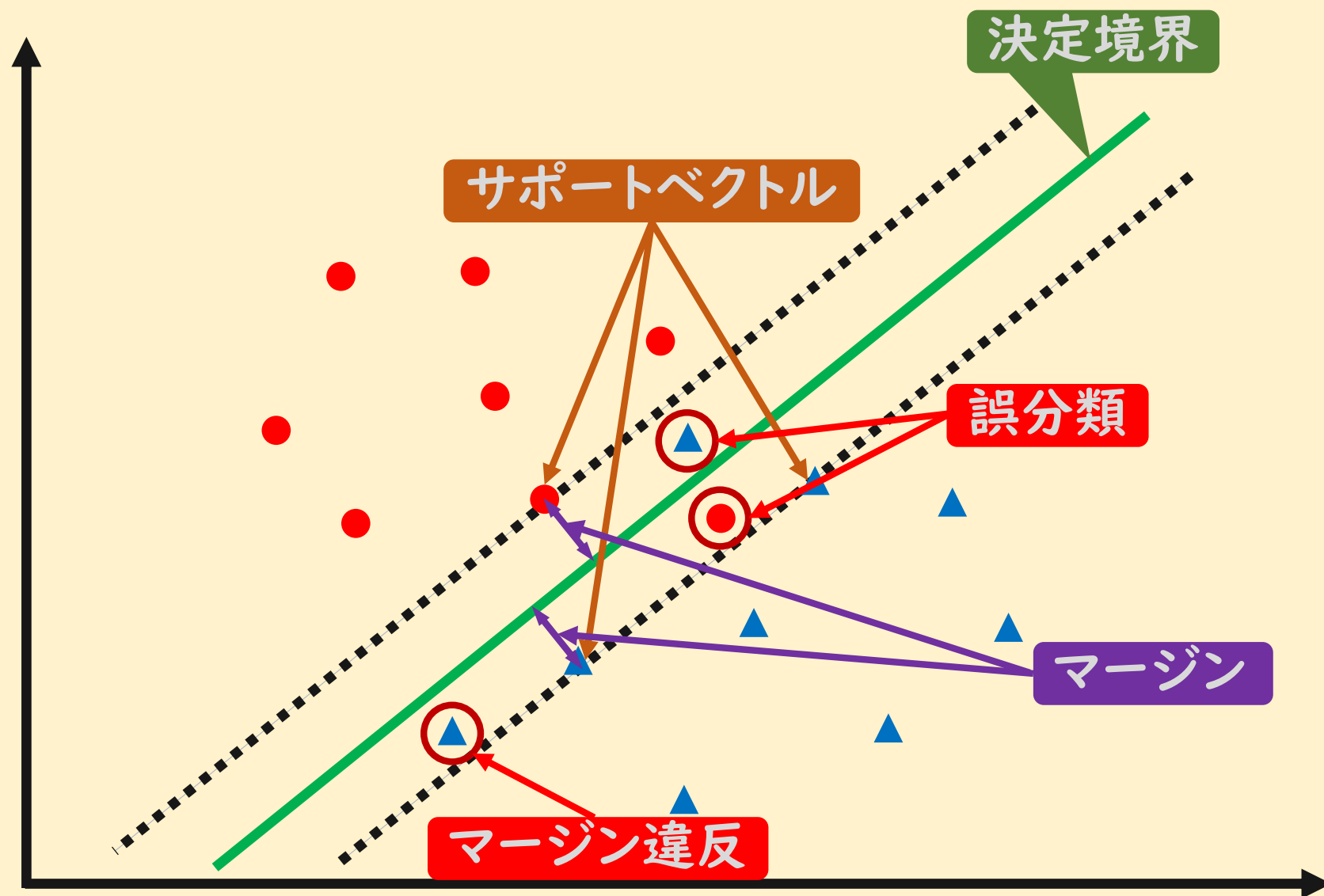
$$\mathbf{w} = \sum_{\substack{p=1 \\ a_p > 0}}^N a_p y^p \mathbf{x}^p$$

$$b = \frac{1}{N} \sum_{p=1}^N \left(y^p - \sum_{\substack{q=1 \\ a_q > 0}}^N a_q y^q (\mathbf{x}^q)^T \mathbf{x}^p \right)$$

ソフトマージン線形SVM

線形SVM(ソフトマージン)の概念

112



マージン最大化(ソフトマージン線形SVM)

113

制約条件付き最適化:

ハードマージン
型との相違

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 + \alpha \sum_{p=1}^N \xi_p$$

$$\text{subject to } y^p (\mathbf{w}^T \mathbf{x}^p + b) \geq 1 - \xi_p \quad \xi_p \geq 0 \quad (p = 1 \sim N)$$

制約条件: マージン内に最小限
のインスタンスしか存在しない

但し, α は誤りの許容度

定式化(ソフトマージン線形SVM)

114

制約条件付き最適化問題:

関数 L の最小化問題

$$L(\mathbf{w}, b, \mathbf{a}, \boldsymbol{\eta}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{p=1}^N a_p \left(y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1 + \xi_p \right)$$

$$+ \alpha \sum_{p=1}^N \xi_p - \sum_{p=1}^N \eta_p \xi_p$$

ハードマージン
型との相違

ソフトマージンの制約ベクトル: $\boldsymbol{\xi} = (\xi_1, \xi_2, \dots, \xi_N)^T$

ラグランジュ乗数ベクトル: $\mathbf{a} = (a_1, a_2, \dots, a_N)^T$

$$\boldsymbol{\eta} = (\eta_1, \eta_2, \dots, \eta_N)^T$$

ラグランジュ未定乗数法の制約条件($p = 1 \sim N$)

$$\left\{ \begin{array}{l} a_p \geq 0 \\ y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1 + \xi_p \geq 0 \\ a_p \{ y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1 + \xi_p \} = 0 \\ \xi_p \geq 0 \\ \eta_p \geq 0 \\ \eta_p \xi_p = 0 \end{array} \right.$$

ラグランジュ関数 L を w, b, ξ_p で各々偏微分し, ゼロとおくと,

$$\frac{\partial L}{\partial w} = w - \sum_{p=1}^N a_p y^p x^p = \mathbf{0}$$

$$\frac{\partial L}{\partial b} = - \sum_{p=1}^N a_p y^p = 0$$

$$\frac{\partial L}{\partial \xi_p} = \alpha - a_p - \eta_p = 0 \quad (p = 1 \sim N)$$

従って,

$$\mathbf{w} = \sum_{p=1}^N a_p y^p \mathbf{x}^p$$

$$\sum_{p=1}^N a_p y^p = 0$$

$$\alpha = a_p + \eta_p = 0 \quad (p = 1 \sim N)$$

これら関数 L へ代入すると,

$$\begin{aligned} L(\mathbf{w}, b, \xi, \mathbf{a}, \mu) &= \frac{1}{2} \|\mathbf{w}\|^2 + \alpha \sum_{p=1}^N \xi_p \\ &\quad - \sum_{p=1}^N a_p (y^p (\mathbf{w}^T \mathbf{x}^p + b) - 1 + \xi_p) - \sum_{p=1}^N \eta_p \xi_p \\ &= -\frac{1}{2} \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q \\ &\quad - \sum_{p=1}^N a_p y^p b + \sum_{p=1}^N a_p + \sum_{p=1}^N (\alpha - a_p - \eta_p) \\ &= -\frac{1}{2} \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q + \sum_{p=1}^N a_p \end{aligned}$$

ハードマージン
型と同一

双対問題(ソフトマージン線形SVM)[4/4]

119

従って, 関数 L の双対問題は, 次式となる

ハードマージン
型と同一

$$\min_a \left(-\frac{1}{2} \sum_{p=1}^N \sum_{q=1}^N a_p a_q y^p y^q (\mathbf{x}^p)^T \mathbf{x}^q + \sum_{p=1}^N a_p \right)$$

$$\text{subject to } \sum_{p=1}^N a_p y^p = 0, \quad 0 \leq a_p \leq \alpha \quad (p = 1 \sim N)$$

ハードマージン
型との相違

最適解(ソフトマージン線形SVM)

120

最適解:

$$\mathbf{w} = \sum_{p=1}^N a_p y^p \mathbf{x}^p$$

$$0 < a_p < \alpha$$

ハードマージン
型との相違

$$b = \frac{1}{N} \sum_{p=1}^N \left(y^p - \sum_{q=1}^N a_q y^q (\mathbf{x}^q)^T \mathbf{x}^p \right)$$

$$0 < a_q < \alpha$$

クラス分類の評価

1. **混同行列** (Confusion Matrix)

2. **正解率** (Accuracy) または **精度** (Precision)

1. 混同行列(2クラス分類の場合)

123

予測 \ 正解	陽性	陰性
	陽性	陰性
陽性	TP (True Positive) 真陽性	FP (False Positive) 偽陽性
陰性	FN (False Negative) 偽陰性	TN (True Negative) 真陰性

2. 正解率(精度)

正解 予測	陽性	陰性
	陽性	陰性
陽性	TP (True Positive) 真陽性	FP (False Positive) 偽陽性
陰性	FN (False Negative) 偽陰性	TN (True Negative) 真陰性

124

- 定義

正しく分類したインスタンスの割合

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- 特徴

陽性と陰性のインスタンス数に偏りがない場合に有効

- 事例

数字の0~9の手書き数字認識

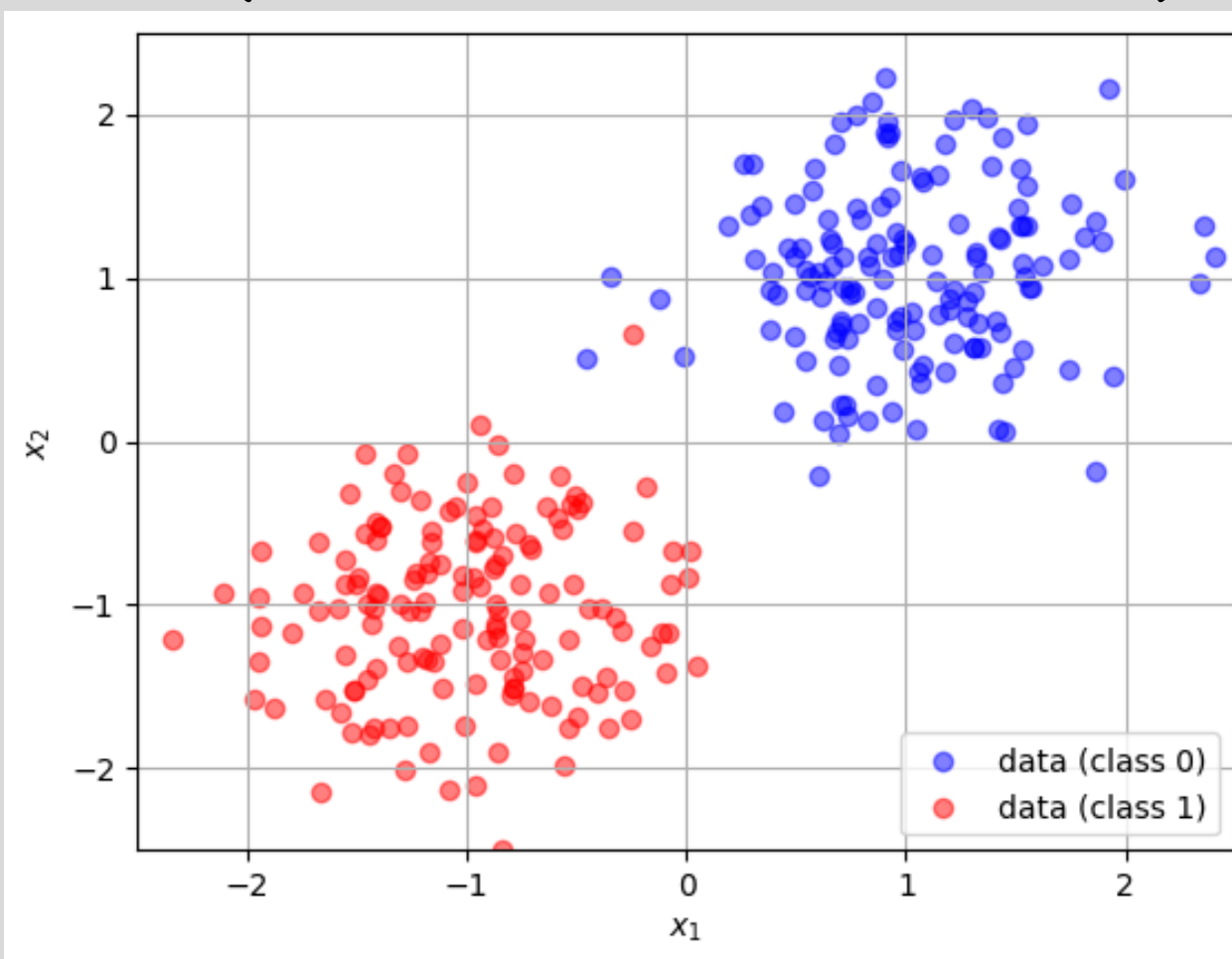
数字の5と5以外の認識 → 偏りが大きい

5のクラスは5以外の
クラスの約9倍

演習

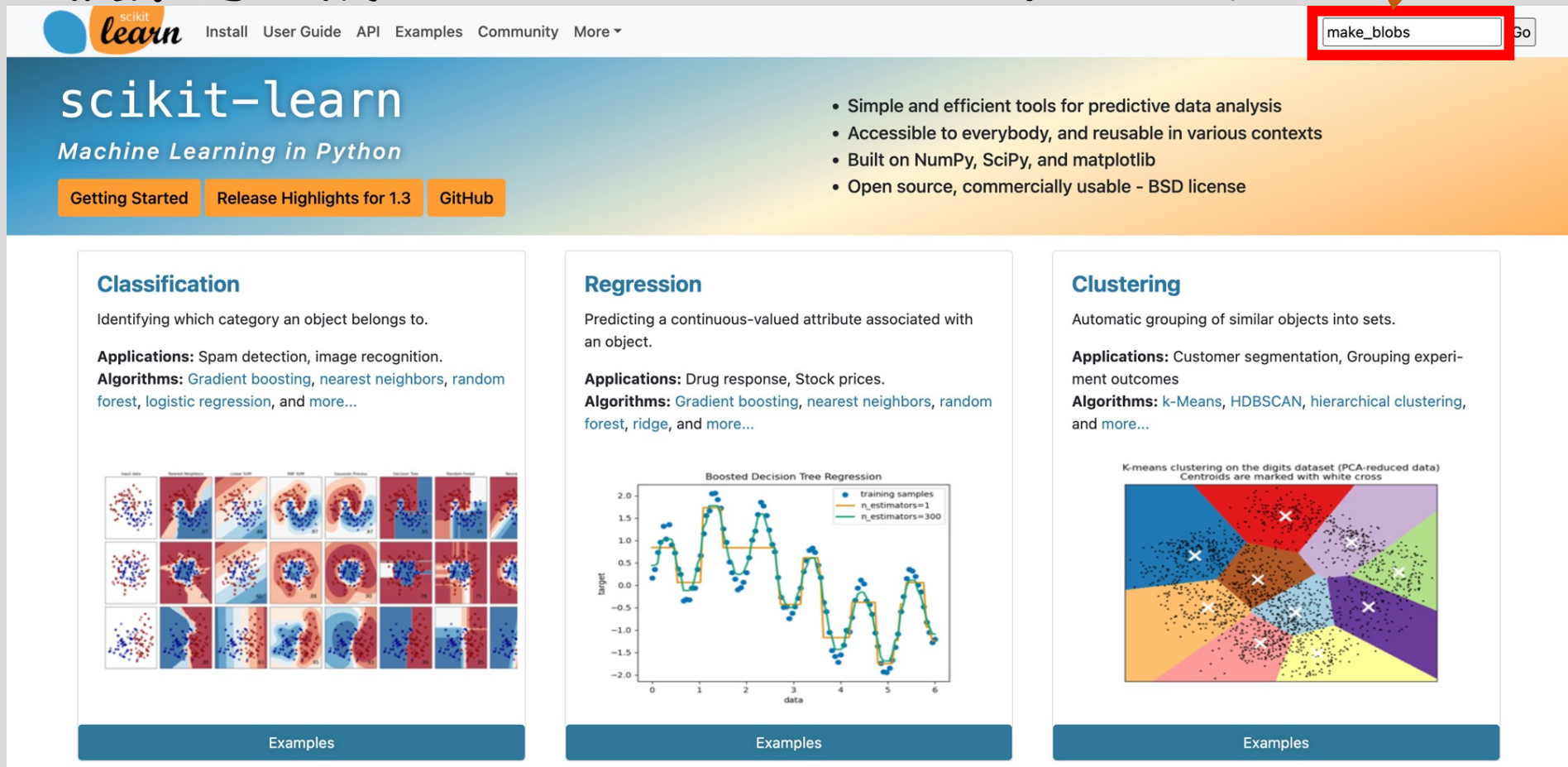
(線形SVMによるクラス分類)

- **scikit-learn**(機械学習ライブラリ)
- **make_blobs**(ガウス分布状のデータ生成)



- **scikit-learnのサイトへ**
検索窓に調べたいクラスやメソッド名を入力

make_blobsと入力



The screenshot shows the scikit-learn website. At the top, there is a navigation bar with links: Install, User Guide, API, Examples, Community, and More. A search bar in the top right corner contains the text 'make_blobs' and a 'Go' button. Below the navigation bar, the scikit-learn logo and tagline 'Machine Learning in Python' are displayed. To the right of the logo, there are three buttons: 'Getting Started', 'Release Highlights for 1.3', and 'GitHub'. Further right, there is a list of features: 'Simple and efficient tools for predictive data analysis', 'Accessible to everybody, and reusable in various contexts', 'Built on NumPy, SciPy, and matplotlib', and 'Open source, commercially usable - BSD license'. Below this, there are three main sections: 'Classification', 'Regression', and 'Clustering'. Each section has a brief description, applications, algorithms, and a grid of example plots. The 'Classification' section shows a 3x3 grid of plots for different classifiers. The 'Regression' section shows a plot of 'Boosted Decision Tree Regression'. The 'Clustering' section shows a plot of 'K-means clustering on the digits dataset (PCA-reduced data)'.

Classification
Identifying which category an object belongs to.
Applications: Spam detection, image recognition.
Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more...

Regression
Predicting a continuous-valued attribute associated with an object.
Applications: Drug response, Stock prices.
Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...

Clustering
Automatic grouping of similar objects into sets.
Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...

出典: scikit-learn (<https://scikit-learn.org/stable/index.html>)

make_blobsの検索結果

128

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)

[Prev](#) [Up](#) [Next](#)

scikit-learn 1.3.1
[Other versions](#)

Please [cite us](#) if you use the software.

[sklearn.datasets.make_blobs](#)
[make_blobs](#)
[Examples using sklearn.datasets.make_blobs](#)

Toggle Menu

sklearn.datasets.make_blobs

```
sklearn.datasets.make_blobs(n_samples=100, n_features=2, *, centers=None, cluster_std=1.0, center_box=(-10.0, 10.0), shuffle=True, random_state=None, return_centers=False)
```

[\[source\]](#)

Generate isotropic Gaussian blobs for clustering.

Read more in the [User Guide](#).

Parameters:

n_samples : *int or array-like, default=100*
If int, it is the total number of points equally divided among clusters. If array-like, each element of the sequence indicates the number of samples per cluster.

Changed in version v0.20: one can now pass an array-like to the `n_samples` parameter

n_features : *int, default=2*
The number of features for each sample.

centers : *int or array-like of shape (n_centers, n_features), default=None*
The number of centers to generate, or the fixed center locations. If n_samples is an int and centers is None, 3 centers are generated. If n_samples is array-like, centers must be either None or an array of length equal to the length of n_samples.

cluster_std : *float or array-like of float, default=1.0*
The standard deviation of the clusters.

center_box : *tuple of float (min, max), default=(-10.0, 10.0)*
The bounding box for each cluster center when centers are generated at random.

shuffle : *bool, default=True*
Shuffle the samples.

random_state : *int, RandomState instance or None, default=None*
Determines random number generation for dataset creation. Pass an int for reproducible output across multiple function calls. See [Glossary](#).

return_centers : *bool, default=False*
If True, then return the centers of each cluster.

New in version 0.23.

Returns:

X : *ndarray of shape (n_samples, n_features)*
The generated samples.

y : *ndarray of shape (n_samples,)*
The integer labels for cluster membership of each sample.

centers : *ndarray of shape (n_centers, n_features)*
The centers of each cluster. Only returned if `return_centers=True`.

引数

- `n_samples`: データの個数
- `n_features`: 特徴量の数
- `centers`: 塊の数, または座標
- `cluster_std`: ばらつきの大きさ (標準偏差)
- `random_state`: 乱数生成のシード値
※ 同じ設定値で再現性が確保

戻り値

- `X`: 入力データ (特徴量, $n_samples \times n_features$)
- `y`: 出力データ (クラス番号, $n_samples$)

make_blobsの使い方(コード)

130

```
from sklearn.datasets import make_blobs  
import pandas as pd
```

```
X, y = make_blobs(n_samples=300, n_features=2,  
centers=[[1, 1], [-1, -1]], cluster_std=0.5,  
random_state=3)
```

特徴量

クラスラベル

サンプル数300個

特徴量2次元

中心座標

乱数シード値

標準偏差
(散らばり具合)

```
df = pd.DataFrame(X)
```

データフレーム化

```
df["target"] = y
```

target指定で
クラスラベルの追加

```
df.head()
```

先頭5行の表示

先頭5行のデータ

0番目サンプル
の2次元特徴量

0番目サンプル
のクラス番号

	0	1	target
0	-0.907366	-1.211396	1
1	2.355963	1.313737	0
2	0.698557	0.042764	0
3	-0.721779	-0.626554	1
4	-0.856158	-0.016763	1

データセットの描画(コード)

132

Notebook形式
の場合は必要

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
df0 = df[df["target"]==0]
```

クラス0はdf0
クラス1はdf1

```
df1 = df[df["target"]==1]
```

散布図の描画

```
plt.scatter(df0[0], df0[1], color="b", alpha=0.5, label="data (class 0)")
```

```
plt.scatter(df1[0], df1[1], color="r", alpha=0.5, label="data (class 1)")
```

```
plt.xlabel("$x_1$")
```

軸ラベルの設定

```
plt.ylabel("$x_2$")
```

凡例の設定(右下)

```
plt.legend(loc="lower right")
```

```
plt.xlim(-2.5,2.5)
```

軸範囲の設定

```
plt.ylim(-2.5,2.5)
```

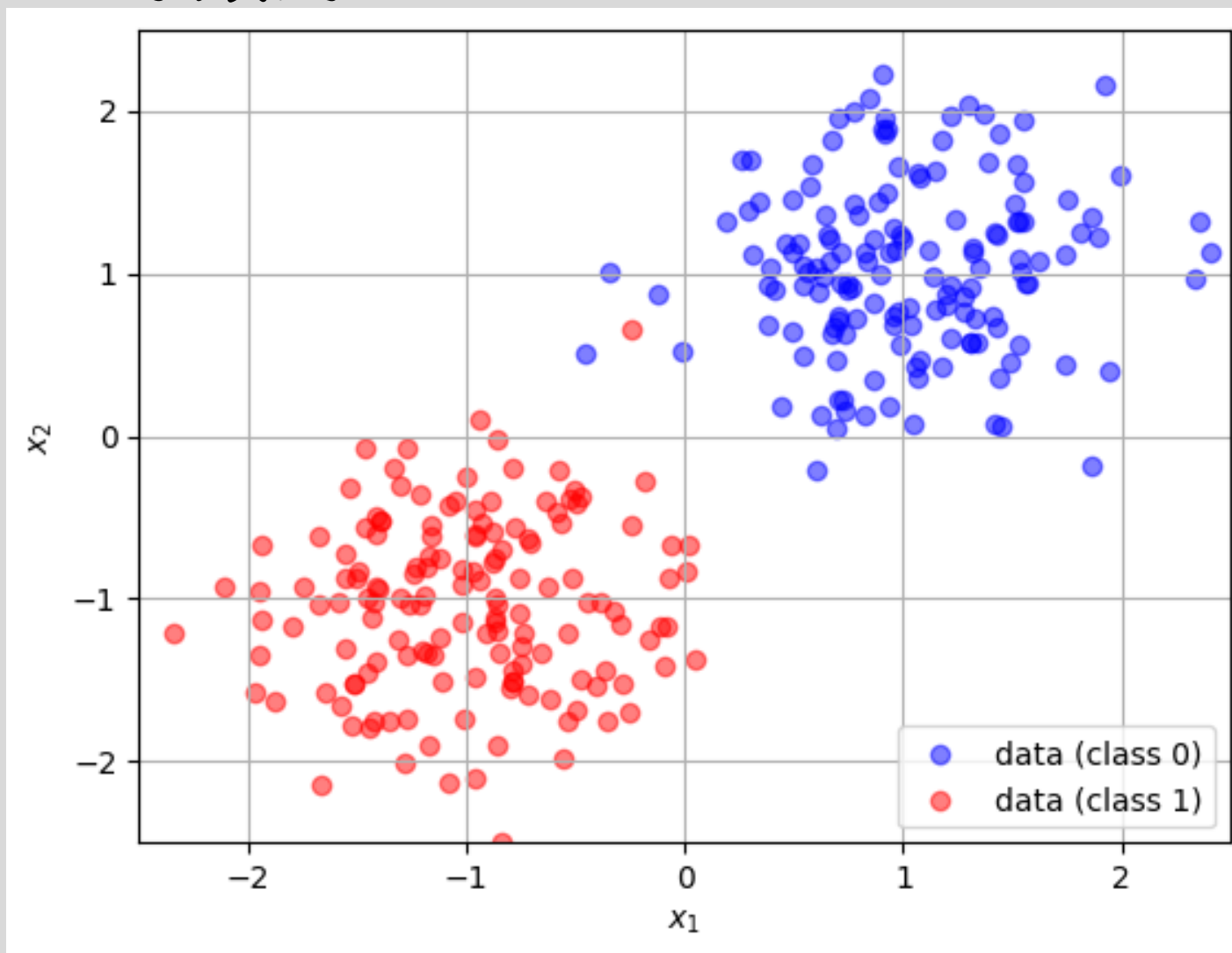
```
plt.grid()
```

グリッドの設定

```
plt.show()
```

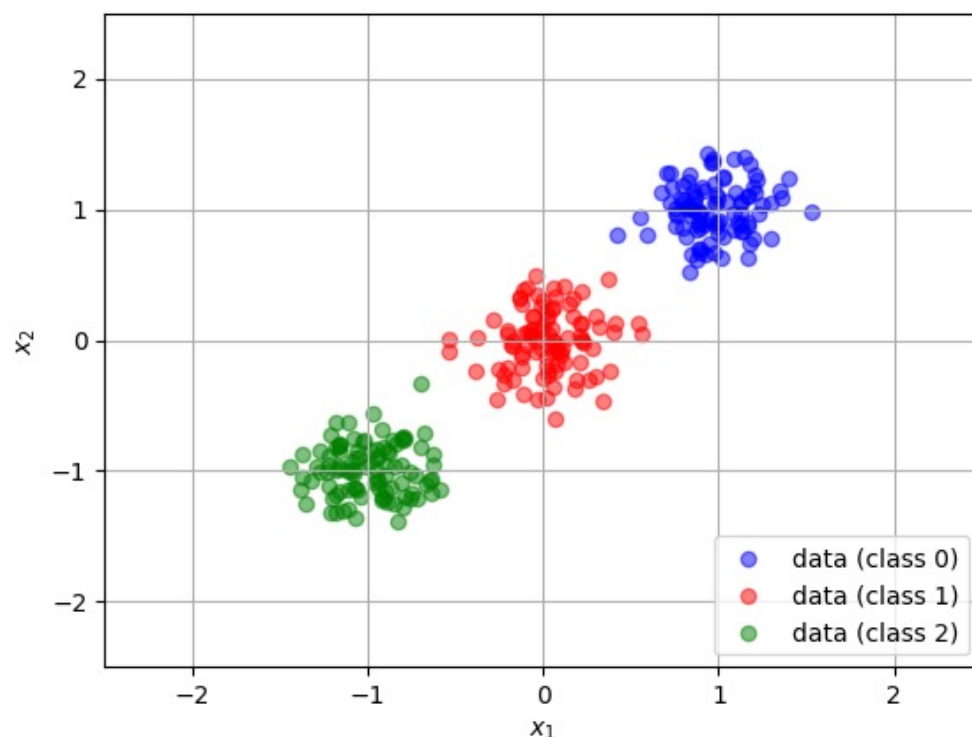
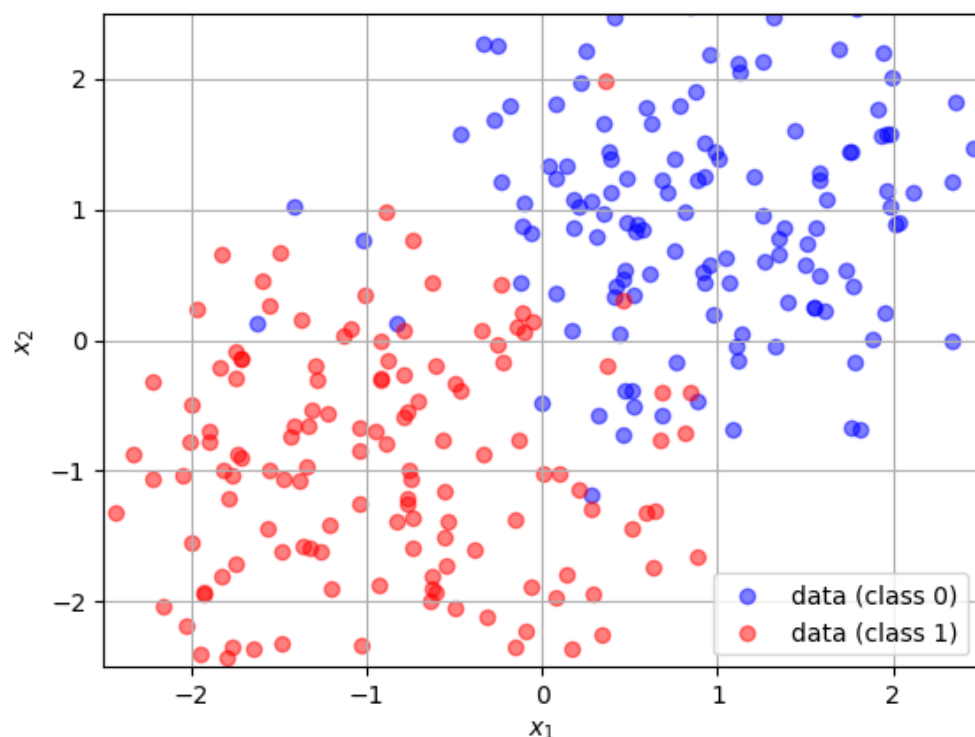
グラフの描画

- 2クラス分類用データセット



他のデータセットの生成

134



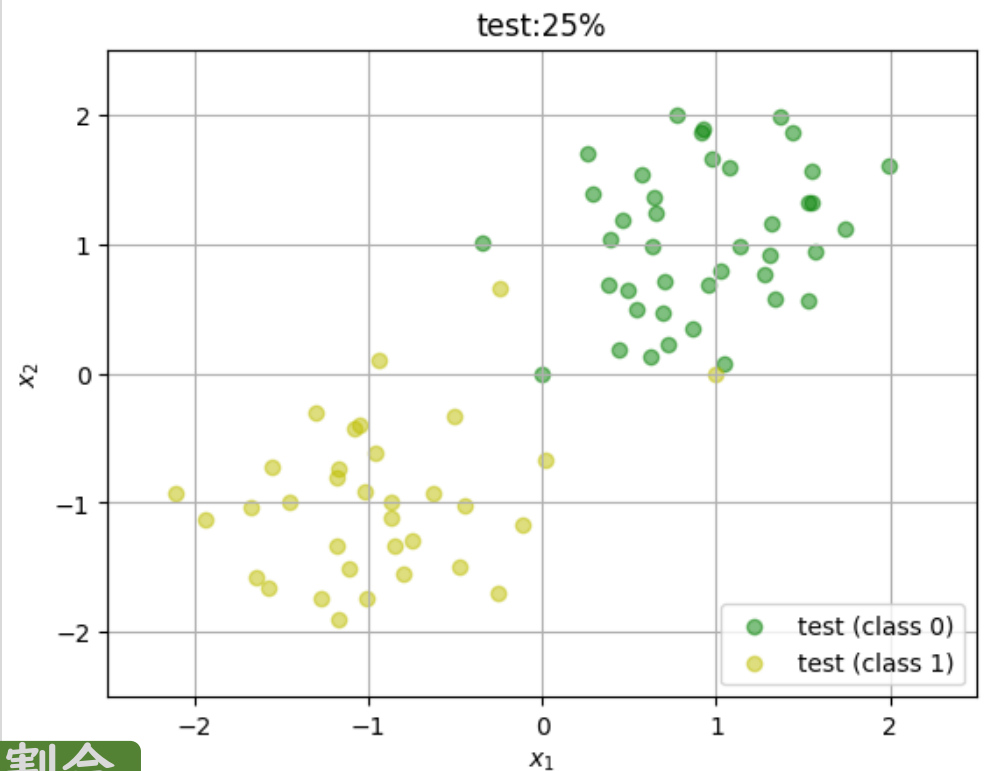
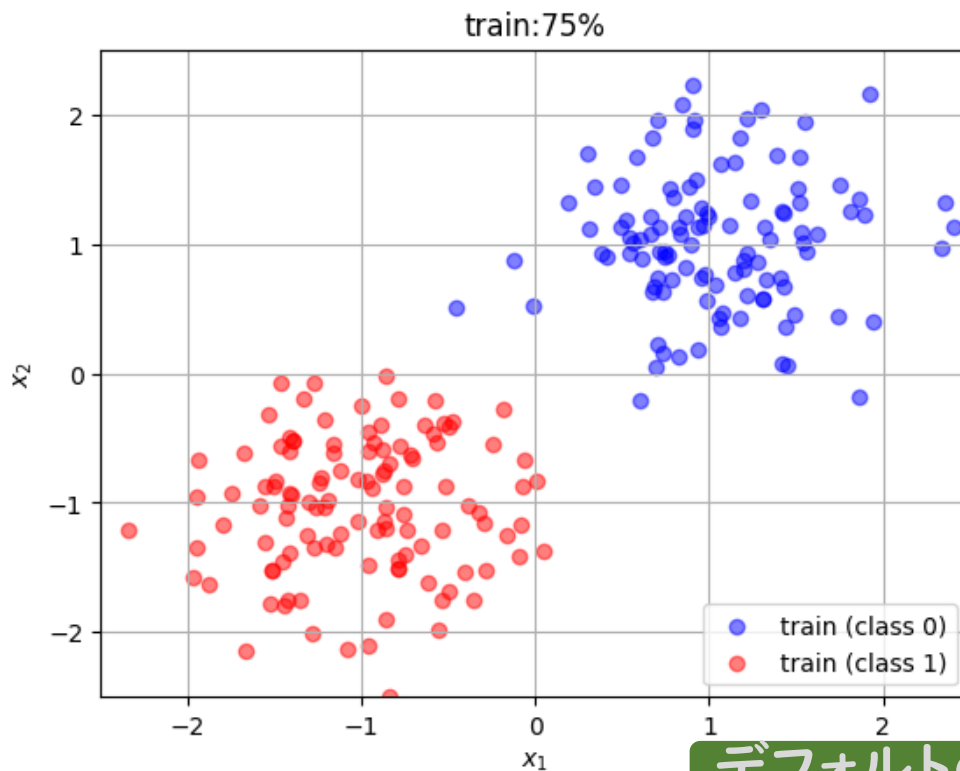
左図: $\text{cluster_std}=0.9$ の場合
(ばらつきが大きな分布)

右図: $\text{centers}=3$ の場合
(クラス数が3つ)

データセットの分割(コード)

135

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
random_state=0)
```



デフォルトの割合

左図: 訓練データ(75%)

右図: テストデータ(25%)

引数

デフォルト値は0.25

- test_size: テストデータのサイズ指定 (0~1.0)
- train_size: 訓練データのサイズ指定 (0~1.0)
- random_state: 乱数生成のシード値

戻り値

- X_train: 訓練用入力データ
- X_test: テスト用入力データ
- y_train: 訓練用出力データ
- y_test: テスト用出力データ

線形SVMの学習(コード)

137

```
from sklearn.svm import SVC
```

Support Vector
Classificationのクラス

線形モデルの指定

```
model = SVC(kernel='linear', C=100)
```

正則化係数
C:大→ハードマージン型
C:小→ソフトマージン型

```
model.fit(X_train, y_train)
```

モデルの学習

テストデータによる評価(コード)

138

```
from sklearn.metrics import accuracy_score
```

クラスラベルの予測

```
pred_test = model.predict(X_test)
```

正解率の計算

```
score = accuracy_score(y_test, pred_test)
```

```
print("正解率 = {0:5.1f}%".format(score*100))
```

```
from sklearn.metrics import confusion_matrix
```

```
print("混同行列=\n{0}".format(confusion_matrix(y_test, pred_test)))
```

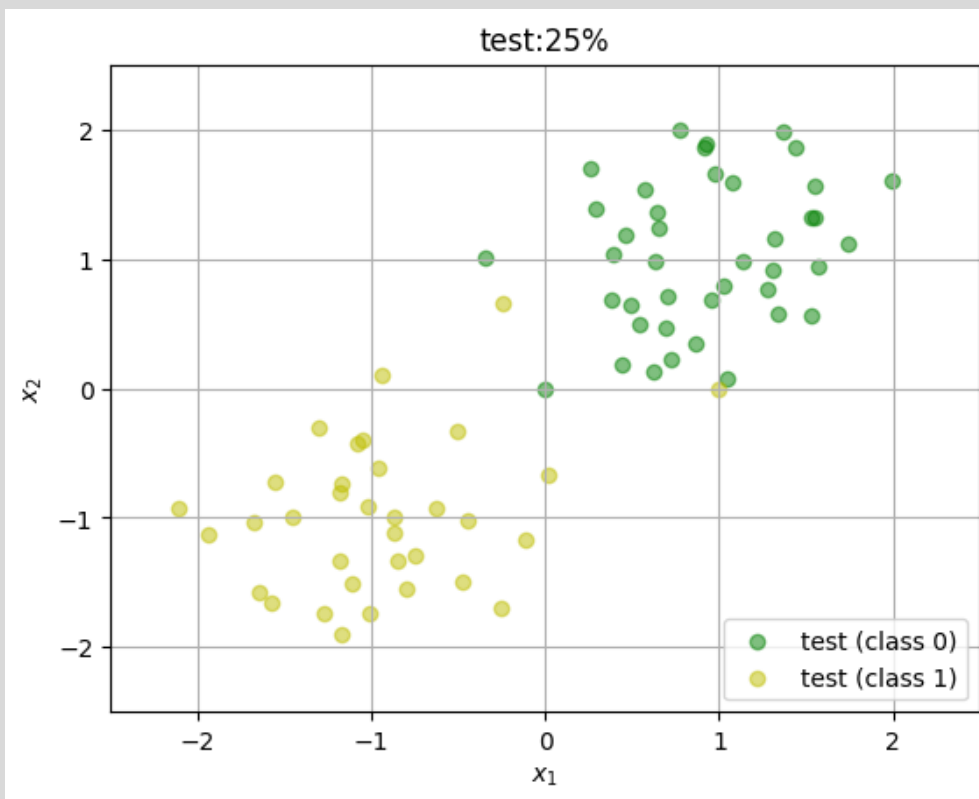
ハードマージン線形SVMの結果[1/2]¹³⁹

誤分類2個

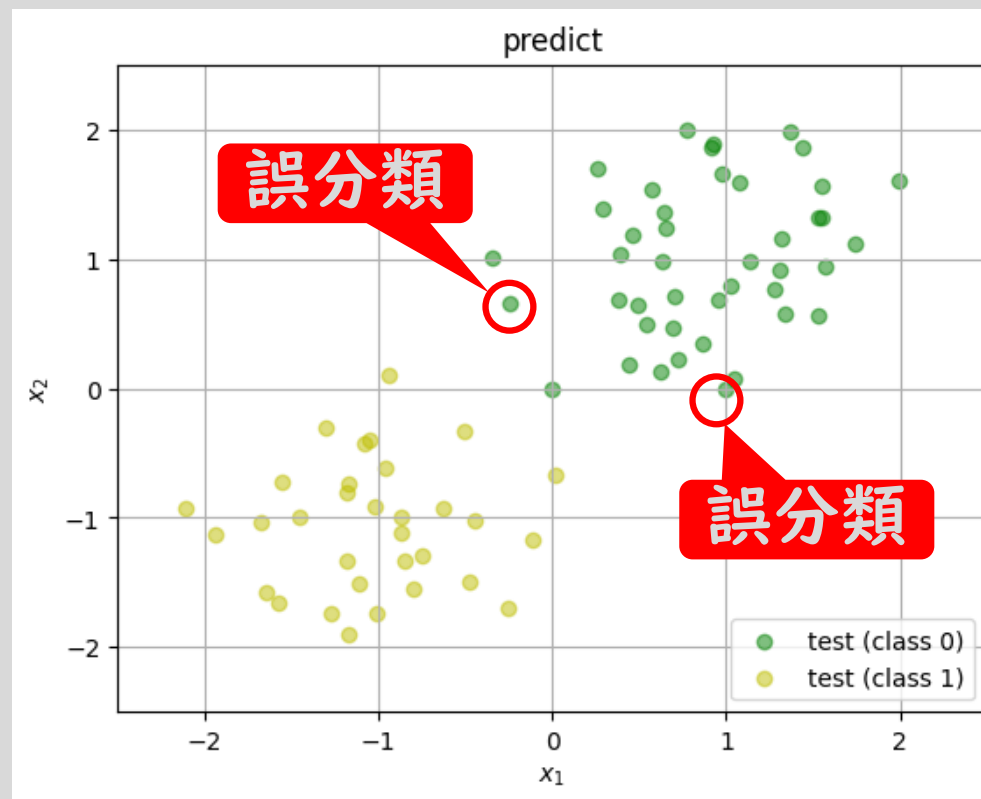
正解率 = 97.3% (=73/75)

混同行列: $\begin{bmatrix} 41 & 0 \\ 2 & 32 \end{bmatrix}$

クラス1の2個
が0に誤分類



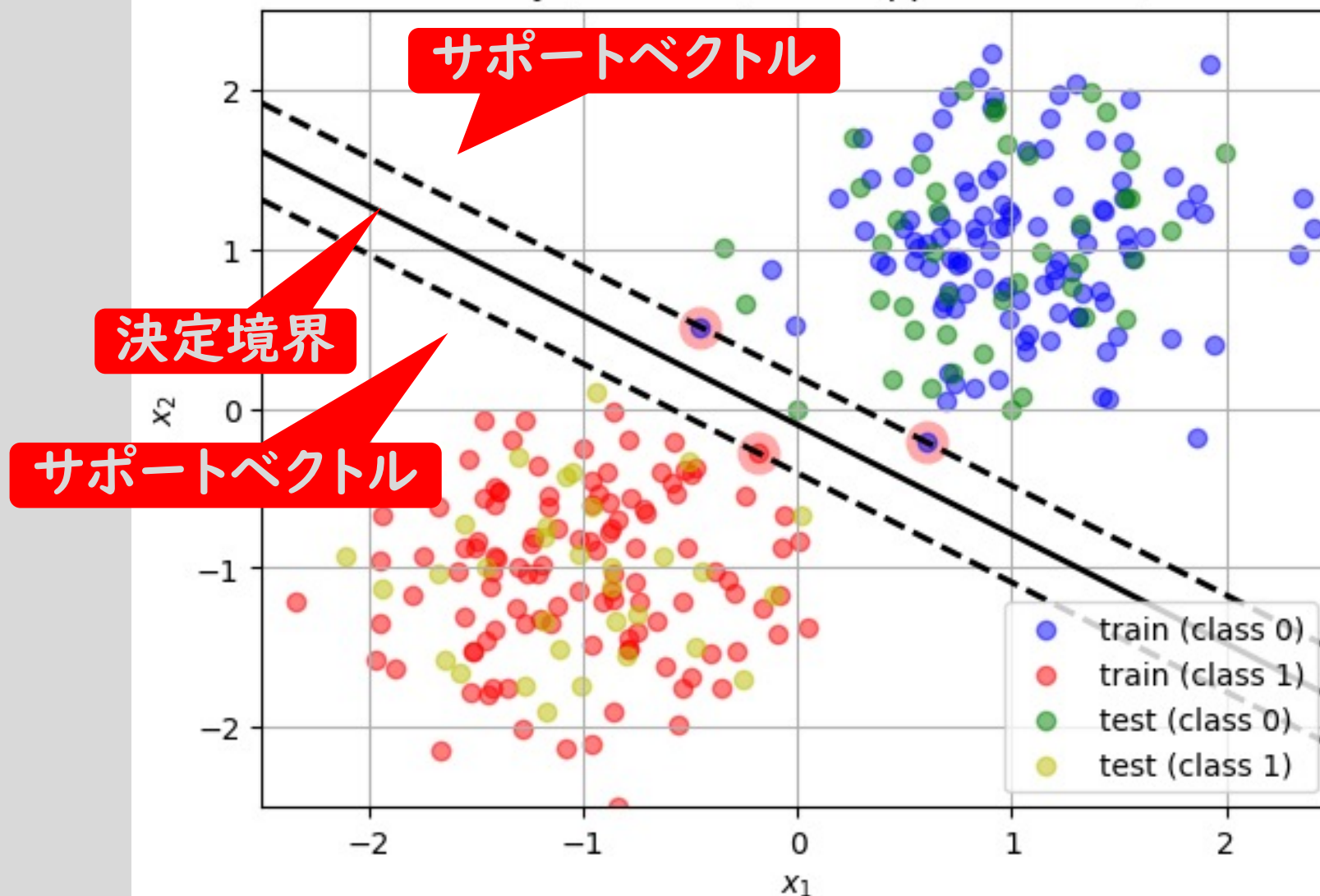
左図: テストデータ



右図: 分類結果

ハードマージン線形SVMの結果[2/2]₁₄₀

Decision Boundary (solid line) and Support Vectors (dotted lines)



決定境界(実線)とサポートベクトル(点線)

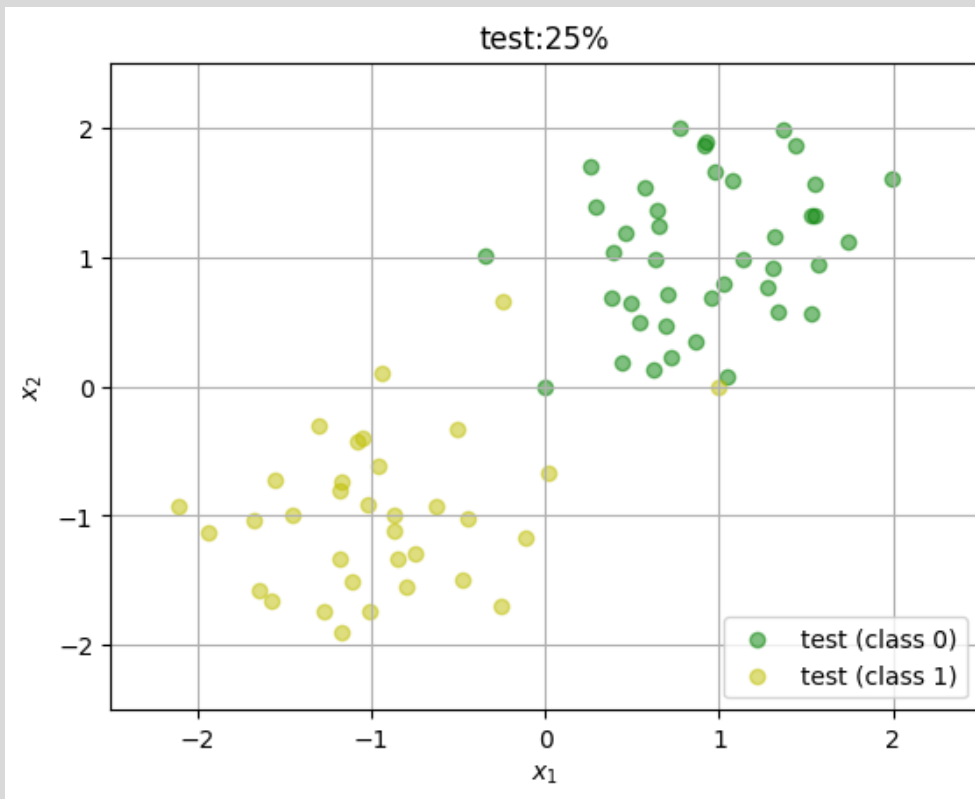
ソフトマージン線形SVMの結果[1/2] 141

誤分類3個

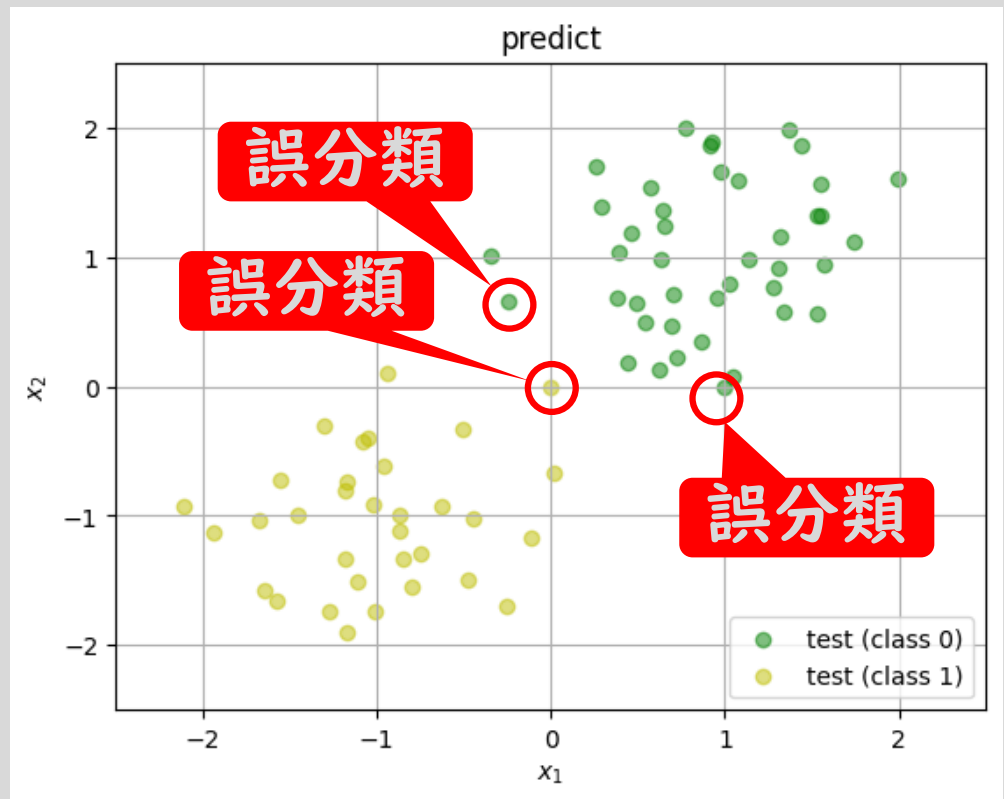
正解率 = 96.0% (=72/75)

混同行列: $\begin{bmatrix} 40 & 1 \\ 2 & 32 \end{bmatrix}$

クラス0の1個が1, クラス1
の2個が0に誤分類



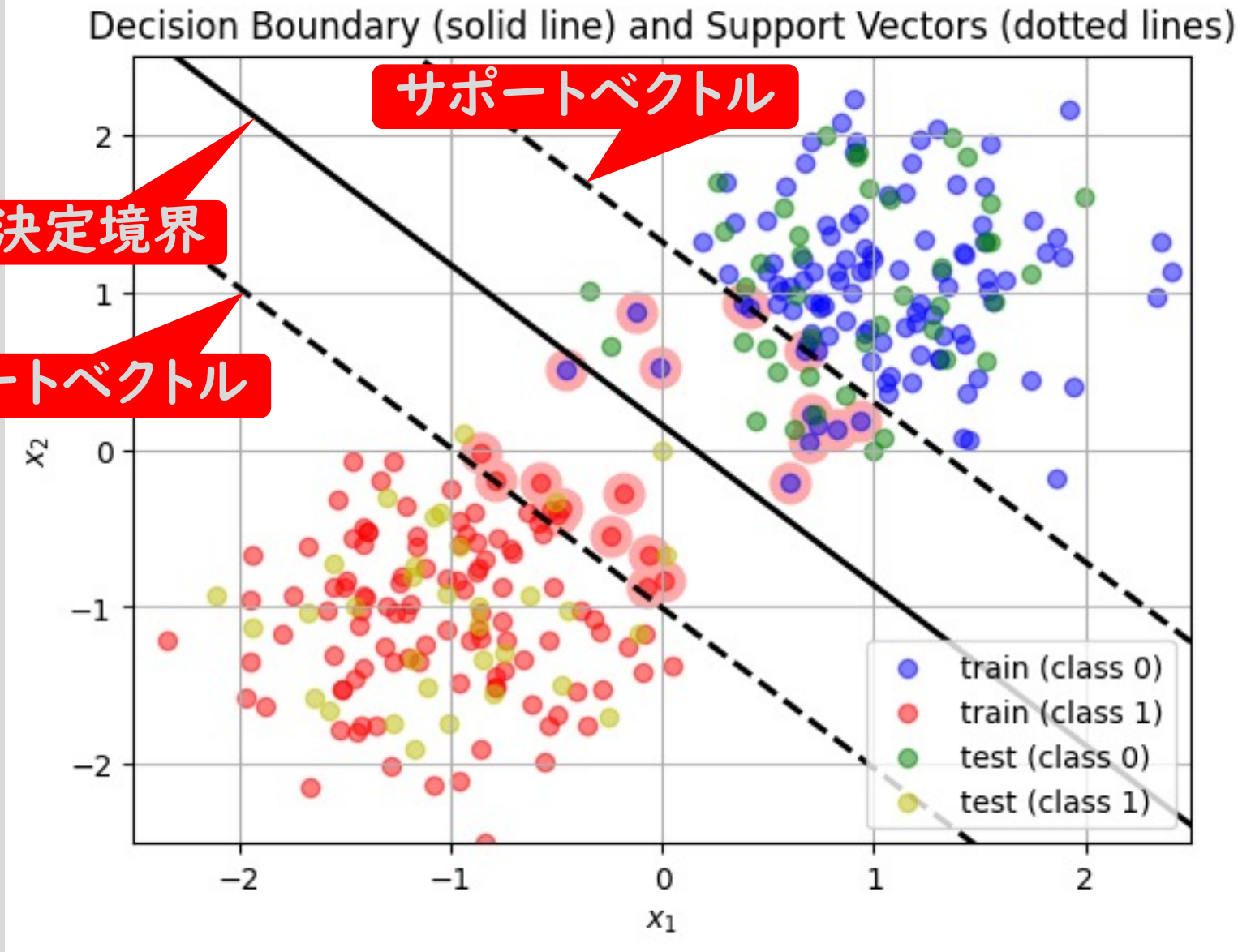
左図: テストデータ



右図: 分類結果

ソフトマージン線形SVMの結果[2/2]

142



決定境界(実線)とサポートベクトル(点線)

決定境界を描く関数(コード)

143

```
import numpy as np
import matplotlib.pyplot as plt
def plot_svc_decision_boundary(svm_clf, xmin, xmax):
    w = svm_clf.coef_[0]
    b = svm_clf.intercept_[0]
    x0 = np.linspace(xmin, xmax, 500)
    decision_boundary = -w[0]/w[1] * x0 - b/w[1]
    margin = 1/w[1]
    gutter_up = decision_boundary + margin
    gutter_down = decision_boundary - margin
    svs = svm_clf.support_vectors_
    plt.scatter(svs[:, 0], svs[:, 1], s=180, facecolors='#FFAAAA')
    plt.plot(x0, decision_boundary, "k-", linewidth=2)
    plt.plot(x0, gutter_up, "k--", linewidth=2)
    plt.plot(x0, gutter_down, "k--", linewidth=2)
```